Driver Fatigue Detection

Final Project Technical Report

Proposed by:

Group 4

Professor:

Adrian Carrio Fernandez

BCSAI: Bachelor of Computer Science & Artificial Intelligence

&

AI: COMPUTER VISION

Madrid, Spain

www.ie.edu

May, 2026

Group 4

Abstract

This project implements a computer vision system that monitors driver fatigue, but only activates after the driver performs a specific hand gesture sequence in front of the camera. The reasoning behind this design is straightforward: a fatigue detector that runs all the time would generate too many false alarms from normal facial movements, so requiring a deliberate activation step makes the system more practical and realistic. We built and compared two full pipelines for each task: a classical one based on handcrafted features and traditional machine learning, and a modern one based on deep learning. On top of both, we also implemented data-free heuristic baselines for gesture recognition and fatigue detection, which turned out to be competitive with or better than the trained models given the size of our dataset. All evaluation is done using leave-one-person-out (LOSO) cross-validation on video data we recorded ourselves inside a parked car.

1. System Overview

The system is always in one of three states: GESTURE WAITING, FATIGUE MONITOR, or ALERT.

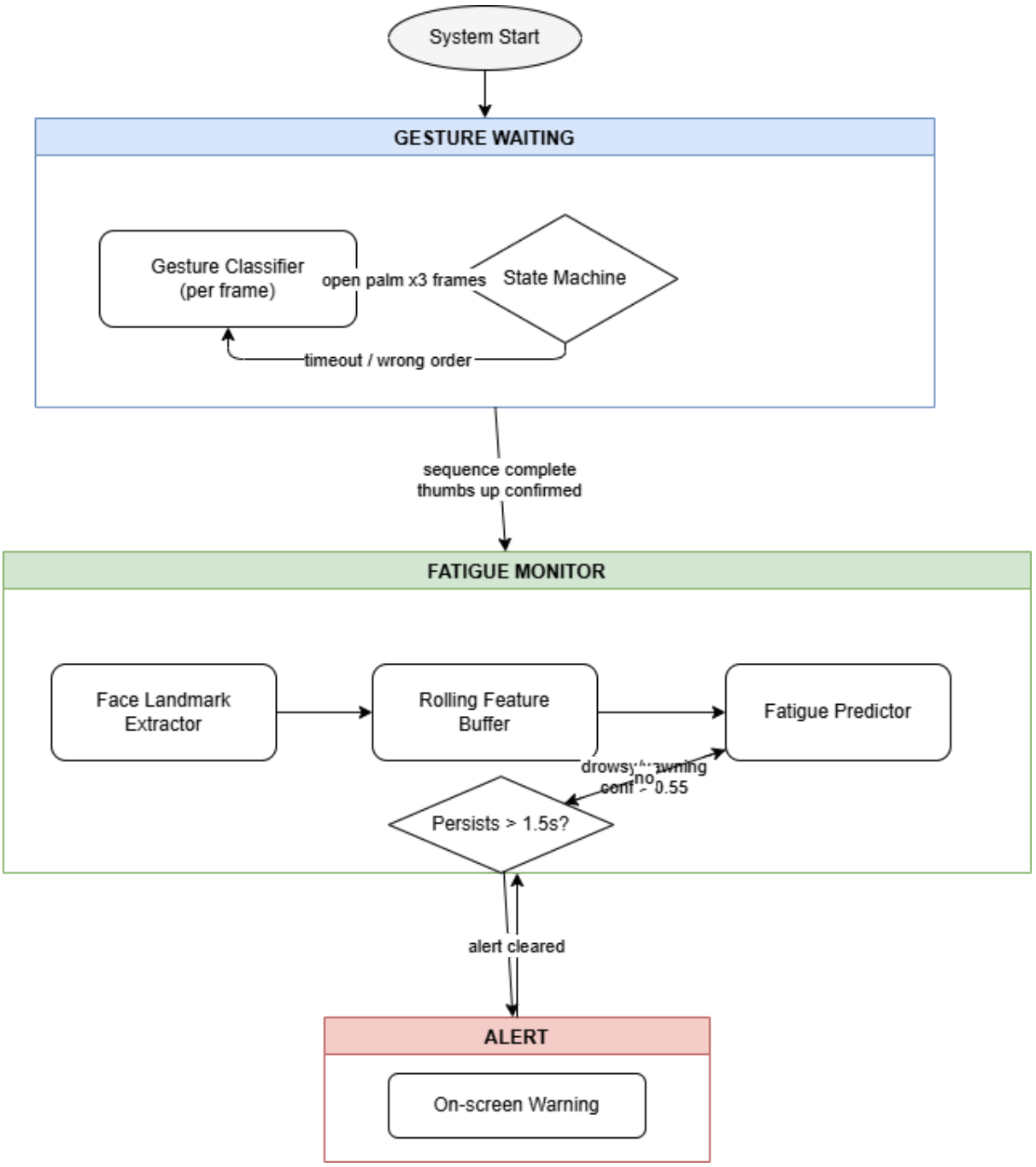
When the program starts it goes into GESTURE WAITING. The camera runs and a gesture classifier looks at each frame to decide if it sees an open palm, a thumbs up, or neither. A state machine watches these predictions and waits for the driver to do the right sequence (open palm first, then thumbs up) within a 5-second window. To avoid triggering on a single bad frame, each gesture has to appear in at least 3 consecutive frames with confidence above 0.6. If the driver completes the sequence in time, the system activates. If the window expires or the gestures come in the wrong order, it resets.
Once activated, the system moves to FATIGUE MONITOR and stops looking at the hands. From this point MediaPipe processes the face on every frame and adds a feature vector to a rolling buffer. A fatigue predictor reads that buffer and labels the driver as alert, drowsy, or yawning. If it keeps predicting drowsy or yawning above 0.55 confidence for more than 1.5 seconds straight, the system moves to ALERT and shows a warning on screen.

For each task we implemented two pipelines:

1. Gesture recognition: a classical SVM and Random Forest trained on 63-dimensional hand landmark vectors, and a MobileNetV3-Small CNN trained on 96x96 hand image crops.
2. Fatigue detection: a classical per-frame SVM and RF with clip-level aggregation, and a 1D Temporal CNN that processes the whole feature sequence at once.

Both tasks also have a data-free heuristic that uses pure geometry or simple thresholds, no training required.

The high-level architecture is illustrated below:



2. Dataset

We recorded all the videos inside a parked car, sitting in the driver seat with the camera on the dashboard. Two subjects took part, each recording multiple clips per category.
For fatigue we recorded 13 fine-grained categories and collapsed them into three coarse classes. Clips of normal alertness, looking around, talking, and phone use are all labelled alert. Clips of slow blinking, eyes closed, head drooping, head tilting, microsleep, and fatigue face are labelled drowsy. Yawning clips form their own class. Transition clips were held out.

For gestures we recorded individual gesture clips for training the per-frame classifier (open palm, thumbs up, random hands, and no hands) and full sequence clips for testing the end-to-end activation: correct sequence, wrong order, too slow, and incomplete.
We did not use any external datasets. With only two subjects the LOSO evaluation is noisy by nature, so we always report per-fold numbers alongside the mean.

3. Feature Extraction

3.1 Hand Features

MediaPipe Hands detects up to one hand per frame. When it finds one, the 21 3D landmarks are normalised and flattened into a 63-dimensional vector. The classical gesture classifier trains directly on these vectors. For the CNN we instead crop a 96x96 patch around the detected hand with a 30% margin and feed the image to the network.

3.2 Face Features

MediaPipe FaceLandmarker gives us three things per frame: 478 face landmarks, 52 blendshape scores, and a transformation matrix. From these we build a 24-dimensional feature vector.
The geometric features come from the raw landmarks. Eye Aspect Ratio (EAR) is computed for the left eye, right eye, and their mean. It measures how open the eye is by comparing vertical to horizontal distances across the eye contour, when it stays low for several seconds the person is likely drowsy. Mouth Aspect Ratio (MAR) does the same thing for the mouth and is useful for detecting yawns. We also include vertical eye and mouth opening gaps, both normalised by face size.
Head pose (pitch, yaw, roll in degrees) comes from the transformation matrix. Head drooping (high pitch) is a strong drowsiness cue.
For blendshapes we use 14 of the 52 available: eyeBlinkLeft, eyeBlinkRight, eyeSquintLeft, eyeSquintRight, eyeLookDownLeft, eyeLookDownRight, jawOpen, mouthClose, mouthFunnel, mouthPucker, browDownLeft, browDownRight, cheekSquintLeft, cheekSquintRight. These are normalised per face by MediaPipe so they transfer better across subjects, and they capture things like squinting that pure geometry misses.

3.3 Clip-Level Aggregate Features

One of the most important design decisions in this project was to compress the per-frame buffer into a 15-dimensional summary vector before classifying fatigue. Fatigue is a temporal pattern, not something we can detect from a single frame. The 15 features capture things like mean and high-percentile eye closure, fraction of the buffer where eyes were clearly closed, longest continuous eye-closed run, blink rate per second, jaw opening levels and their longest burst, head pitch variability, and peak MAR. Together they describe what happened over the last few seconds rather than what the current frame looks like.

4. Gesture Activation

4.1 Classical Classifier

We trained an SVM with an RBF kernel (C=4.0) and a Random Forest with 400 trees on the 63-dimensional landmark vectors. Both use balanced class weights and sit inside a scikit-learn Pipeline with a StandardScaler. The three classes are open_palm, thumbs_up, and negative.

Model	Person 1	Person 2	Mean
SVM	0.813	0.800	0.807
Random Forest	0.865	0.750	0.808
CNN	0.695	0.702	0.699

These numbers look reasonable, but we can see a real deployment problem here:. With only two training subjects, both the SVM and CNN overfit to those specific hands. When we tested on a third person, the SVM would output negative with 99% confidence even on a clearly raised palm. This is exactly why we built the gesture heuristic.

4.2 Gesture Heuristic

The gesture heuristic in src/gestures/heuristic.py is a rule-based classifier that uses only landmark geometry (no training data needed). It follows the same philosophy as our fatigue heuristic: geometry that is defined the same way for everyone generalises better than a learned model trained on two people.

It first normalises the landmarks relative to the wrist position and hand size. Left hands are mirrored before measuring so the same rules apply to both. Then for each finger it computes an extension ratio (how far the fingertip is from the wrist compared to the MCP joint), and checks whether the finger is pointing upward in the image.

The logic is simple. Open palm fires when the four main fingers are all extended and pointing up. Thumbs up fires when the thumb is clearly pointing up and all four fingers are curled. Everything else is negative. Confidence is computed as a soft combination of per-finger evidence, and it gets pushed to 0.90 when the gesture is clean and unambiguous. This is what the real-time demo uses by default because it works on any hand without needing training data.

4.3 Modern Classifier (CNN)

For the modern pipeline we fine-tuned MobileNetV3-Small (pretrained on ImageNet) on the 96x96 hand crops. It scores lower than both classical models with a mean F1 of 0.699. The likely reason is that MobileNetV3-Small has 1.7 million parameters and simply does not have enough data from two subjects to adapt its ImageNet features to our specific hands and lighting.

4.4 State Machine

The state machine enforces the sequential constraint on top of whichever classifier is chosen. It has three states: IDLE, GOT_GESTURE_1, and ACTIVATED. It counts how many consecutive frames agree on the same label above the confidence threshold (default 0.6). Once 3 consecutive frames confirm a gesture, the state advances. A 5-second window limits how long the driver has to complete the sequence before it resets.
For the trained classifiers in the real-time demo we also apply a negative discount of 0.35: when MediaPipe has already confirmed a hand is present, the negative class probability gets multiplied by 0.35. This is needed because the training negative class mixes both random gestures and frames with no hand at all, which pushes the classifier to over-predict negative when a real hand is actually there.

4.5 End-to-End Results

Model	Precision	Recall	F1	Accuracy
SVM + State Machine	0.788	0.867	0.825	0.927
CNN + State Machine	0.719	0.767	0.742	0.893

The SVM wins here. The 0.867 recall means it correctly activates on 86.7% of valid sequences, and the 0.927 accuracy means it barely ever triggers on sequences it should reject.

5. Fatigue Detection

5.1 Classical Per-Frame Classifier

We trained the same SVM and Random Forest setup on the 24-dimensional face features. Per-frame performance is low, which was expected.

Model	Mean Accuracy	Macro-F1
SVM	0.509	0.403
Random Forest	0.648	0.502

A single frame is just not enough information to tell drowsy from alert.

5.2 Temporal Aggregation

We applied two simple aggregation strategies over a sliding window of frames. Mean probability averages the softmax outputs and takes the argmax. Window vote takes the majority of per-frame labels. Both improve on raw per-frame prediction, confirming that temporal context is the key ingredient.

5.3 Temporal Aggregation

The modern pipeline trains a 1D CNN directly on the per-frame feature sequences instead of classifying per frame and aggregating after. The input is the same 24-dimensional feature vector at each frame so the comparison with classical is fair.
The architecture has four Conv1d blocks (widths 64, 64, 128, 128 with kernel sizes 5, 5, 3, 3), each followed by BatchNorm and ReLU. The output goes through masked global average pooling, where a binary mask ensures that zero-padded frames do not contribute to the pooled representation. Sequences are all padded or truncated to 64 frames. A dropout layer (p=0.3) and a linear head produce the final 3-class output.
Training ran for 60 epochs with AdamW and a cosine annealing schedule. A weighted sampler and class-weighted loss handle the class imbalance (roughly 58% drowsy, 32% alert, 10% yawning).
We used three augmentations at training time only: small Gaussian noise on the features to simulate MediaPipe jitter, a random temporal shift of up to 4 frames before padding, and per-feature channel dropout at probability 0.05. These pushed mean macro-F1 from 0.714 to 0.830, which was the biggest single gain in the whole pipeline.

5.4 Heuristic Baseline

The fatigue heuristic is a simple two-rule classifier with no trainable parameters.
For yawning: if the 90th percentile of jawOpen over the buffer is above 0.25, the clip is labelled yawning. We use the 90th percentile because a yawn is a short burst and the mean is too diluted to catch it reliably.
For drowsy vs alert: a drowsiness score is computed as blink_mean divided by 0.45, and an alert score as one minus that. Whichever is higher wins. The boundary works out to around a blink_mean of 0.225, which sits well below drowsy clips (average around 0.40) and well above alert ones (around 0.18).
Since MediaPipe normalises blendshapes per face this rule works the same way regardless of who is in front of the camera. It reached mean macro-F1 of 0.877.

5.5 Aggregate Classifier

The aggregate classifier trains a Logistic Regression on the 15 clip-level summary features. We chose logistic regression on purpose , with roughly 212 clips and two subjects in LOSO, a complex model would just overfit. A Random Forest was also tested and the one with better LOSO macro-F1 was used as the deployment model. This reached mean macro-F1 of 0.876 with probabilities that are better calibrated than the heuristic, making the alert threshold behave more predictably. It is the default fatigue model in the demo.

5.6 Ensemble

We blended the heuristic and the aggregate classifier. The default approach uses the aggregate classifier everywhere except when the heuristic strongly predicts yawning, in which case the heuristic takes over. A 50/50 blend was also tested. The ensemble got mean macro-F1 of 0.843, slightly below both components individually, which means they are mostly capturing the same signal and not complementing each other.

5.7 Full Results

Model	Mean Accuracy	Macro-F1
RF + Window Vote (classical)	0.693	0.504
Temporal CNN (modern, augmented)	0.844	0.830
Heuristic (data-free)	0.887	0.877
Aggregate-CLF (deployment)	0.877	0.876
Ensemble	0.858	0.843

6. Real-Time System

The real-time demo runs from scripts/run_realtime.py. It reads from the webcam or a video file, resizes frames to 720px wide, and passes each frame with a timestamp to the RealtimeFatigueSystem orchestrator.

The system is frame-rate agnostic, the fatigue buffer adjusts based on a target FPS so it always covers roughly the same amount of real time regardless of camera speed. The MediaPipe extractors are loaded once at startup and reused.

The defaults are the gesture heuristic for activation and the aggregate classifier for fatigue. The trained models are available via command-line flags but are not recommended for live use on new subjects because of the overfitting issue.

The HUD shows a colour-coded banner at the top (grey, green, or red depending on state), gesture progress, probability bars for gesture and fatigue classes, a flashing alert with a seconds counter, and a live FPS display in the corner. The --output flag saves everything to an MP4 file, which is how we made the demo video.

7. Discussion

The result that stands out the most is that temporal context is what makes fatigue detection actually work. The per-frame RF gets a mean macro-F1 of 0.502. Switching to the 15-feature clip-level summary pushes any model past 0.87. The temporal CNN shows the same thing from a different angle, it processes sequences directly and gets 0.830, which is much better than the 0.502 per-frame baseline even though both use the same input features.

The other surprising result is that the data-free heuristics matched or beat every trained model on both tasks. For fatigue the heuristic gets 0.877 with no training at all. For gestures it works reliably on new subjects where the trained SVM completely collapses. The reason is the same in both cases: with only two training subjects, learned models inevitably pick up person-specific patterns that do not transfer to anyone else. Rules based on normalised geometry do not have this problem.

The accuracy gap between person1 and person2 in the fatigue results (0.935 vs 0.819 for the aggregate classifier) comes directly from having only two LOSO folds. Each fold trains on one person and tests on the other, so whatever is specific to the training subject does not carry over. More subjects would close this gap.

Limitations

We only had two subjects recorded in a single session, so the dataset is limited in terms of lighting conditions, face variation, and the range of actual drowsiness patterns. The fatigue states were performed rather than naturally induced, so there is a chance models learned exaggerated poses rather than subtle real-world drowsiness cues.

8. Conclusion

We built a driver fatigue detection system with gesture-based activation and compared classical and modern pipelines for both tasks. The system activates reliably using the geometric gesture heuristic and detects fatigue at mean macro-F1 of 0.876 with the aggregate classifier.
The two main lessons from this project are that temporal aggregation matters more than model architecture for fatigue detection, and that on a small dataset with two subjects, well-designed geometric rules can outperform trained models. These findings shaped our deployment choices directly: the heuristic handles gesture activation and the aggregate classifier handles fatigue, while the trained pipelines serve as the comparison baseline in the evaluation.

9. Team Contributions

This project was a collaborative effort by all five members of Group 4. Work was distributed across the main project areas: data collection, classical pipeline, modern pipeline, real-time system, and report writing, with everyone contributing across multiple stages and reviewing each other's work in regular team check-ins.

Member	Main contributions
Gonzalo Fernandez de Cordoba	Set up the repository structure and reproducibility pipeline, designed and implemented the real-time orchestrator and state machine, built the per-frame and clip-level feature extractors, trained and evaluated the classical and modern fatigue models (SVM/RF, temporal CNN, aggregate classifier, ensemble), implemented both data-free heuristics, ran the LOSO experiments and figure generation
Mia Dragovic	Helped record and label the in-car video dataset, contributed to the fatigue feature engineering (EAR, MAR, head-pose features), assisted with experiments on the temporal CNN augmentation strategy, and helped edit and structure the report.

Group 4

Leena El Barq Helped record and label the in-car video dataset for both subjects, contributed to the gesture-recognition pipeline (classical SVM/RF training and evaluation), and reviewed the report for clarity and technical accuracy.

Omar El Hajj Contributed to the gesture CNN pipeline (MobileNetV3-Small fine-tuning and end-to-end sequence evaluation), helped with figure generation and the comparison tables, and proofread the report.

Ruben Grande Helped record the in-car dataset, contributed to the state-machine design and tested the real-time activation flow under different conditions, and reviewed the discussion and limitations sections of the report.

All members took part in the final integration testing and contributed feedback on design decisions throughout the project